Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет: Инженерно-экономический

Кафедра: Экономической информатики

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2
на тему:
« Компонентная модель и управление зависимостями в Jakarta EE»

Выполнил Летко А. С.
гр. 377901

Проверил Русецкий А.Д.

Минск 2026

Цель

Цель работы: изучить компонентную модель Jakarta EE, освоить механизмы управления зависимостями (CDI), реализовать бизнес-логику с использованием сессионных EJB-компонентов (Stateless, Stateful, Singleton), построить REST API для взаимодействия с клиентом, а также освоить методы диагностики и исправления проблем кодировки UTF-8.

Задачи

Задачи лабораторной работы:

1 Создать структуру Maven-проекта для Jakarta EE.

2 Реализовать Singleton EJB для хранения глобального списка анекдотов.

3 Реализовать Stateful EJB для хранения истории просмотров анекдотов конкретным пользователем.

4 Реализовать Stateless EJB для бизнес-логики выбора случайного анекдота.

5 Использовать CDI для:

— Создания продюсера (Producer) для инициализации списка анекдотов.

— Внедрения зависимостей (Inject).

— Генерации события (JokeShownEvent) при показе анекдота.

— Создания наблюдателя (Observer) для обновления глобальной статистики на основе события.

— Реализации интерсептора (@Logged) для логирования вызовов.

6 Построить REST-слой, предоставляющий эндпоинты:

— /api/joke/random — получение случайного анекдота.

— /api/joke/history — получение истории просмотров пользователя.

— /api/joke/all — получение всех доступных анекдотов.

7 Произвести сборку проекта и деплой на сервер приложений GlassFish

8 Устранить возможные проблемы с кодировкой UTF-8.

Архитектура приложения

Разрабатываемое приложение следует трехуровневой архитектуре Jakarta EE:

— Слой представления (REST): Принимает HTTP-запросы, делегирует их EJB-компонентам и возвращает ответ;

— Слой бизнес-логики (EJB): Реализует бизнес-правила: хранение данных (Singleton), управление состоянием сессии (Stateful), выборку данных (Stateless);

— Инфраструктурный слой (CDI): Обеспечивает внедрение зависимостей, генерацию событий, логирование и фабрикацию объектов.

Практическая часть

Индивидуальное задание:
ТЕМА 14. Сервис анекдотов
A) Формулировка для преподавателя
Тема: Сервис анекдотов с использованием всех типов EJB и CDI.
Требования:
1. Singleton — хранит список анекдотов.
2. Stateful — хранит историю пользователя.
3. Stateless — выбирает случайный анекдот.
4. CDI требования: • продюсер списка анекдотов; • событие JokeShownEvent; • наблюдатель, обновляющий глобальную статистику; • инъекция генератора анекдотов; • интерсептор @Logged.
5. REST: /joke/random, /joke/history, /joke/all.

Методические акценты: • Отличный пример взаимодействия трёх типов EJB + CDI.
B) Формулировка для студента
1. Singleton — хранит анекдоты.
2. Stateful — хранит историю.
3. Stateless — выбирает анекдот.
4. CDI: событие, наблюдатель, продюсер.
5. REST: /joke/random, /joke/history, /joke/all.

Результаты работы:

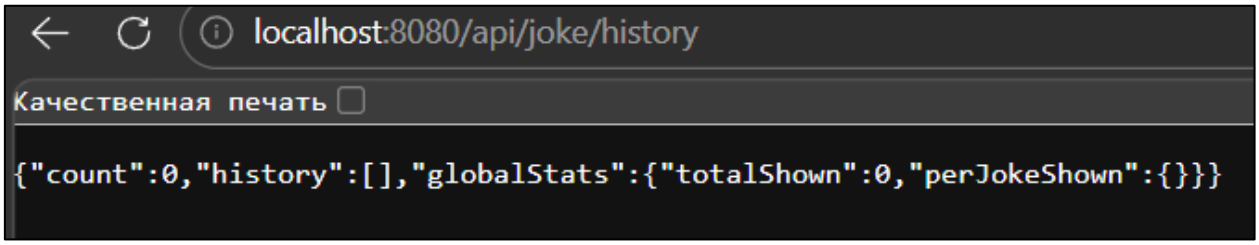

Рисунок 1 – Просмотр истории до вызова случайной шутки

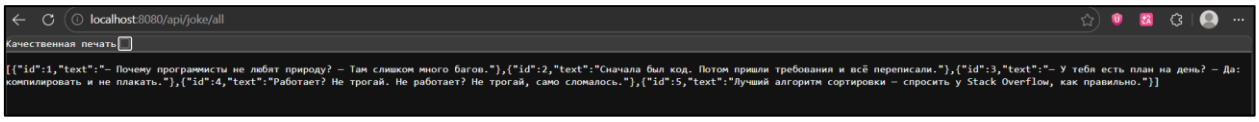

Рисунок 2 – Просмотр всех шуток

localhost:8080/api/joke/random

Качественная печать

{"id":3,"text":"— У тебя есть план на день? — Да: компилировать и не плакать."}

Рисунок 3 – Вызов добавления случайной шутки

localhost:8080/api/joke/history

Качественная печать

{"count":1,"history":[{"id":3,"text":"— У тебя есть план на день? — Да: компилировать и не плакать."}],"globalStats":{"totalShown":1,"perJokeShown":{"3":1}}}

Рисунок 4 – Просмотр истории после добавления одной шутки

localhost:8080/api/joke/random

Качественная печать

{"id":3,"text":"— У тебя есть план на день? — Да: компилировать и не плакать."}

Рисунок 5 – Добавление 2-ой случайной шутки

localhost:8080/api/joke/random

Качественная печать

{"id":1,"text":"— Почему программисты не любят природу? — Там слишком много багов."}

Рисунок 6 – Добавление 3-ей случайной шутки

localhost:8080/api/joke/random

Качественная печать

{"id":5,"text":"Лучший алгоритм сортировки — спросить у Stack Overflow, как правильно."}

Рисунок 7 – Добавление 4-ой случайной шутки

localhost:8080/api/joke/random

Качественная печать

{"id":5,"text":"Лучший алгоритм сортировки — спросить у Stack Overflow, как правильно."}

Рисунок 8 – Добавление 5-ой случайной шутки

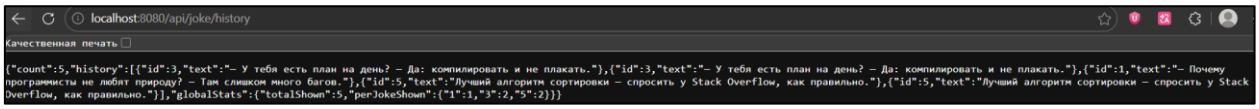

Рисунок 9 – Просмотр истории после добавления 5-ти шуток

Листинг кода

JokeListProducer:

```
package edu.lab.cdi;

import edu.lab.model.Joke;
import jakarta.enterprise.inject.Produces;
import jakarta.inject.Named;

import java.util.List;

public class JokeListProducer {

    @Produces
    @Named("jokes")
    public List<Joke> produceJokes() {
        return List.of(
                new Joke(1, "— Почему программисты не любят природу? — Там слишком много
багов."),
                new Joke(2, "Сначала был код. Потом пришли требования и всё
переписали."),
                new Joke(3, "— У тебя есть план на день? — Да: компилировать и не
плакать."),
                new Joke(4, "Работает? Не трогай. Не работает? Не трогай, само
сломалось."),
                new Joke(5, "Лучший алгоритм сортировки — спросить у Stack Overflow, как
правильно.")
        );
    }
}
```

JokeShownEvent:

```
package edu.lab.cdi;

import edu.lab.model.Joke;

public class JokeShownEvent {

    private final Joke joke;

    public JokeShownEvent(Joke joke) {
        this.joke = joke;
    }

    public Joke getJoke() {
        return joke;
    }
}
```

JokeShownObserver:

```
package edu.lab.cdi;

import edu.lab.ejb.JokeStatisticsBean;
```

```
import jakarta.enterprise.event.Observes;
import jakarta.inject.Inject;

public class JokeShownObserver {

    @Inject
    private JokeStatisticsBean statistics;

    public void onJokeShown(@Observes JokeShownEvent event) {
        statistics.onJokeShown(event.getJoke());
    }
}
```

Logged:

```
package edu.lab.cdi;

import jakarta.interceptor.InterceptorBinding;

import java.lang.annotation.Inherited;
import java.lang.annotation.Retention;
import java.lang.annotation.Target;

import static java.lang.annotation.ElementType.METHOD;
import static java.lang.annotation.ElementType.TYPE;
import static java.lang.annotation.RetentionPolicy.RUNTIME;

@InterceptorBinding
@Inherited
@Target({TYPE, METHOD})
@Retention(RUNTIME)
public @interface Logged {
}
```

LoggedInterceptor:

```
package edu.lab.cdi;

import jakarta.interceptor.AroundInvoke;
import jakarta.interceptor.Interceptor;
import jakarta.interceptor.InvocationContext;

import java.util.Arrays;
import java.io.Serializable;

@Logged
@Interceptor
public class LoggedInterceptor implements Serializable {

    private static final long serialVersionUID = 1L;

    @AroundInvoke
    public Object log(InvocationContext ctx) throws Exception {
        long start = System.currentTimeMillis();
        String cls = ctx.getMethod().getDeclaringClass().getSimpleName();
        String method = ctx.getMethod().getName();
        Object[] params = ctx.getParameters();
        System.out.println("[LOGGED]  ->  "  +  cls  +  "."  +  method  +  "("  +
Arrays.toString(params) + ")");
        try {
            Object result = ctx.proceed();
            long took = System.currentTimeMillis() - start;
            System.out.println("[LOGGED] <- " + cls + "." + method + " took " + took +
"ms");
            return result;
        } catch (Exception e) {
```

```
            long took = System.currentTimeMillis() - start;
            System.out.println("[LOGGED] !! " + cls + "." + method + " failed after " +
took + "ms: " + e);
            throw e;
        }
    }
}
```

JokeGeneratorBean:

```
package edu.lab.ejb;

import edu.lab.cdi.Logged;
import edu.lab.model.Joke;
import edu.lab.service.JokeGenerator;
import jakarta.ejb.Stateless;
import jakarta.inject.Inject;

@Stateless
public class JokeGeneratorBean implements JokeGenerator {

    @Inject
    private RandomJokePickerBean picker;

    @Override
    @Logged
    public Joke nextJoke() {
        return picker.pickRandom();
    }
}
```

JokeRepositoryBean:

```
package edu.lab.ejb;

import edu.lab.model.Joke;
import jakarta.annotation.PostConstruct;
import jakarta.ejb.Singleton;
import jakarta.inject.Inject;
import jakarta.inject.Named;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

@Singleton
public class JokeRepositoryBean {

    private final List<Joke> jokes = new ArrayList<>();

    @Inject
    @Named("jokes")
    private List<Joke> initialJokes;

    @PostConstruct
    void init() {
        jokes.clear();
        if (initialJokes != null) {
            jokes.addAll(initialJokes);
        }
    }

    public List<Joke> getAll() {
        return Collections.unmodifiableList(jokes);
    }
}
```

JokeStatisticsBean:

```
package edu.lab.ejb;

import edu.lab.model.Joke;
import jakarta.ejb.Singleton;

import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicInteger;

@Singleton
public class JokeStatisticsBean {

    private final AtomicInteger totalShown = new AtomicInteger(0);
    private final ConcurrentHashMap<Integer, AtomicInteger> perJokeShown = new
ConcurrentHashMap<>();

    public void onJokeShown(Joke joke) {
        totalShown.incrementAndGet();
        if (joke != null) {
            perJokeShown.computeIfAbsent(joke.getId(), k -> new AtomicInteger(0))
                    .incrementAndGet();
        }
    }

    public Map<String, Object> getStats() {
        Map<Integer, Integer> perJoke = perJokeShown.entrySet().stream()
                .collect(java.util.stream.Collectors.toMap(
                        Map.Entry::getKey,
                        e -> e.getValue().get()
                ));

        return Map.of(
                "totalShown", totalShown.get(),
                "perJokeShown", perJoke
        );
    }
}
```

RandomJokePickerBean:

```
package edu.lab.ejb;

import edu.lab.model.Joke;
import jakarta.ejb.Stateless;
import jakarta.inject.Inject;

import java.util.List;
import java.util.concurrent.ThreadLocalRandom;

@Stateless
public class RandomJokePickerBean {

    @Inject
    private JokeRepositoryBean repository;

    public Joke pickRandom() {
        List<Joke> jokes = repository.getAll();
        if (jokes.isEmpty()) {
            return null;
        }
        int idx = ThreadLocalRandom.current().nextInt(jokes.size());
        return jokes.get(idx);
    }
```

```
}
```

UserJokeHistoryBean:

```
package edu.lab.ejb;

import edu.lab.model.Joke;
import jakarta.ejb.Stateful;

import java.io.Serializable;
import java.util.ArrayList;
import java.util.List;

@Stateful
public class UserJokeHistoryBean implements Serializable {

    private static final long serialVersionUID = 1L;

    private final List<Joke> history = new ArrayList<>();

    public void add(Joke joke) {
        if (joke != null) {
            history.add(joke);
        }
    }

    public List<Joke> getHistory() {
        return new ArrayList<>(history);
    }

    public void clear() {
        history.clear();
    }
}
```

Joke:

```
package edu.lab.model;

import java.io.Serializable;

public class Joke implements Serializable {

    private static final long serialVersionUID = 1L;

    private int id;
    private String text;

    public Joke() {
    }

    public Joke(int id, String text) {
        this.id = id;
        this.text = text;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getText() {
        return text;
```

```
    }

    public void setText(String text) {
        this.text = text;
    }
}
```

JokeResource:

```
package edu.lab.rest;

import edu.lab.cdi.JokeShownEvent;
import edu.lab.cdi.Logged;
import edu.lab.ejb.JokeRepositoryBean;
import edu.lab.ejb.JokeStatisticsBean;
import edu.lab.ejb.UserJokeHistoryBean;
import edu.lab.model.Joke;
import edu.lab.service.JokeGenerator;
import jakarta.enterprise.context.SessionScoped;
import jakarta.enterprise.event.Event;
import jakarta.inject.Inject;
import jakarta.ws.rs.GET;
import jakarta.ws.rs.Path;
import jakarta.ws.rs.Produces;
import jakarta.ws.rs.core.MediaType;
import jakarta.ws.rs.core.Response;

import java.io.Serializable;
import java.util.List;
import java.util.Map;

@SessionScoped
@Path("joke")
@Produces(MediaType.APPLICATION_JSON)
public class JokeResource implements Serializable {

    private static final long serialVersionUID = 1L;

    @Inject
    private JokeRepositoryBean repository;

    @Inject
    private UserJokeHistoryBean history;

    @Inject
    private JokeGenerator generator;

    @Inject
    private Event<JokeShownEvent> jokeShownEvent;

    @Inject
    private JokeStatisticsBean statistics;

    @GET
    @Path("all")
    public List<Joke> all() {
        return repository.getAll();
    }

    @GET
    @Path("random")
    @Logged
    public Response random() {
        Joke joke = generator.nextJoke();
        if (joke == null) {
            return Response.status(Response.Status.NOT_FOUND)
```

```
                    .entity(Map.of("error", "No jokes available"))
                    .build();
        }
        history.add(joke);
        jokeShownEvent.fire(new JokeShownEvent(joke));
        return Response.ok(joke).build();
    }

    @GET
    @Path("history")
    public Map<String, Object> history() {
        List<Joke> jokes = history.getHistory();
        return Map.of(
                "history", jokes,
                "count", jokes.size(),
                "globalStats", statistics.getStats()
        );
    }
}
```

RestApplication:

```
package edu.lab.rest;

import jakarta.ws.rs.ApplicationPath;
import jakarta.ws.rs.core.Application;

@ApplicationPath("api")
public class RestApplication extends Application {
}
```

Beans.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="https://jakarta.ee/xml/ns/jakartaee"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/beans_4_0.xsd"
       bean-discovery-mode="all">
    <interceptors>
        <class>edu.lab.cdi.LoggedInterceptor</class>
    </interceptors>
</beans>
```

glassfish-web.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE glassfish-web-app PUBLIC "-//GlassFish.org//DTD GlassFish Application Server
3.1 Servlet 3.0//EN"
        "http://glassfish.org/dtds/glassfish-web-app_3_0-1.dtd">
<glassfish-web-app>
    <context-root>/</context-root>
</glassfish-web-app>
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/web-app_5_0.xsd"
         version="5.0">
    <display-name>Mini Poll App</display-name>
</web-app>
```

docker-compose.yml:

```
services:
```

```
  glassfish:
    build: .
    image: mini-poll-app:glassfish7
    container_name: mini-poll-app
    ports:
      - "8080:8080"   # HTTP (app: http://localhost:8080/api/poll/...)
      - "4848:4848"   # Admin Console
```

Dockerfile:

```
# Stage 1: build WAR
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app

COPY pom.xml .
RUN mvn -B dependency:go-offline -q

COPY src ./src
RUN mvn -B package -DskipTests -q

# Stage 2: GlassFish 7 runtime
FROM eclipse-temurin:17-jdk
ENV GLASSFISH_VERSION=7.0.25
ENV GLASSFISH_HOME=/opt/glassfish

# Install GlassFish 7
RUN apt-get update -y && apt-get install -y --no-install-recommends \
    curl unzip \
    && curl -sL -o /tmp/glassfish.zip \
    "https://download.eclipse.org/ee4j/glassfish/glassfish-${GLASSFISH_VERSION}.zip" \
    && unzip -q /tmp/glassfish.zip -d /opt \
    && mv /opt/glassfish7 ${GLASSFISH_HOME} \
    && rm /tmp/glassfish.zip \
    && apt-get purge -y curl unzip \
    && apt-get autoremove -y \
    && rm -rf /var/lib/apt/lists/*

# domain1 is already included in the GlassFish distribution

# Deploy WAR to autodeploy
COPY --from=build /app/target/mini-poll-app-*.war \
    ${GLASSFISH_HOME}/glassfish/domains/domain1/autodeploy/mini-poll-app.war

EXPOSE 8080 4848

CMD ["/opt/glassfish/bin/asadmin", "start-domain", "-v"]
```

pom.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>edu.lab</groupId>
    <artifactId>mini-poll-app</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>war</packaging>
    <name>Mini Poll App</name>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
```

```
        <jakarta.jakartaee.version>10.0.0</jakarta.jakartaee.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>jakarta.platform</groupId>
            <artifactId>jakarta.jakartaee-api</artifactId>
            <version>${jakarta.jakartaee.version}</version>
            <scope>provided</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-war-plugin</artifactId>
                <version>3.4.0</version>
                <configuration>
                    <failOnMissingWebXml>false</failOnMissingWebXml>
                </configuration>
            </plugin>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.11.0</version>
                <configuration>
                    <source>17</source>
                    <target>17</target>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```